

Relatório 2ª Fase | Laboratórios de Informática III
Grupo 41 | 2025/2026

Tomás Ferreira (A110182)
Afonso Barros (A112178)
Lucas Pinto (A111442)

Índice

1. Introdução 3
2. Desenvolvimento 3
   2.1. Sistema 3
   2.2. Discussão 5
      2.2.1. Fluxo do Programa 5
      2.2.2. *Queries* 5
      2.2.3. Otimizações 9
      2.2.4. Testes 10
      2.2.5. Programa Interativo 10
      2.2.6. Métodos de Encapsulamento 11
      2.2.7. Dificuldades Sentidas 11
3. Conclusão 11
4. Anexos 12

1. Introdução

O presente relatório aborda a 2ª fase do projeto da Unidade Curricular de Laboratórios de Informática 3 realizado no ano 25/26 na Universidade do Minho.

O principal objetivo deste trabalho consiste em armazenar e organizar dados de forma modular e garantindo o seu encapsulamento, bem como implementar mecanismos que permitam a realização de operações de pesquisa, associação de forma rápida e minimizando o uso de memória. Nesta fase em específico foi pedido que se implementasse as *queries* 4, 5 e 6 e o programa interativo.

2. Desenvolvimento

O desenvolvimento deste projeto centrou-se na implementação de um sistema modular, capaz de realizar o *parsing* eficiente de ficheiros *CSV* e estruturar a informação em memória, de forma eficiente, para responder a um conjunto de interrogações (*queries*). O sistema aplicado foi pensado para garantir o encapsulamento e suportar diferentes modos de execução: programa--principal, programa-testes e programa-interativo. No modo principal (*batch*), o programa recebe como argumentos os caminhos para o *dataset* e para o ficheiro com os comandos a executar, processa os dados e gera os respetivos resultados em ficheiros individuais na diretoria de saída.

2.1. Sistema

Nesta fase optámos por manter a estrutura geral do sistema concebido na primeira fase, mantendo o objetivo de garantir a separação de responsabilidades entre os diferentes módulos (modularidade) e assegurar o encapsulamento. Foram apenas modificados alguns módulos os quais serão referidos no seguinte texto. O sistema encontra-se dividido em dois componentes principais:

Processamento e Validação dos *Datasets*

Nesta componente, decidimos manter a organização que já tínhamos implementado na primeira fase dentro da pasta entidades. Continuamos a definir as estruturas de dados das entidades apenas nos ficheiros .c, expondo apenas as funções necessárias para as manipular (como os getters, setters e funções de criar/libertar). O objetivo foi garantir que as estruturas se mantêm 'opacas' para o resto do programa. Desta forma, asseguramos que nenhum outro módulo mexe diretamente na memória das entidades.

Na pasta gestores, optámos por manter um gestor para cada entidade, assim como o gestor global, de forma a concentrar toda a informação num único sítio. Esta organização facilitou imenso o desenvolvimento, pois permite passar apenas um argumento entre os módulos para ter acesso a tudo, além de tornar a limpeza de memória muito mais simples e centralizada.

No que toca ao processamento de dados, optámos por introduzir algumas mudanças na estrutura de ficheiros. Criámos uma diretoria IO porque achámos que seria mais lógico e explícito ter num só sítio tudo o que lida com *input* e *output*. Dentro desta pasta encontra-se, por exemplo, a pasta *parsers*, que valida os diferentes campos de uma entidade com recurso ao módulo validações

e, no caso de tudo estar válido, utiliza os outros módulos para criar a entidade e inseri-la no respetivo gestor.

Esta fase ocorre apenas no início do programa, lendo cada *dataset* uma única vez para garantir que toda a informação necessária às *queries* fica imediatamente disponível em memória.

Output e *Queries*

Dentro do módulo de queries temos o interpretador do sistema, que recebe o gestor global e contém a lógica para percorrer o ficheiro de comandos. Aqui fizemos uma alteração importante: em vez de ser a query a escrever diretamente no ficheiro, o interpretador invoca a respetiva query e recebe o seu resultado em formato de texto (algo que foi útil para o modo interativo). É depois o próprio interpretador que recorre ao módulo de output (que definimos dentro da pasta IO) para escrever esse resultado no ficheiro commandX.txt. Desta forma, isolámos a lógica de escrita em ficheiro.
Este módulo não interage com os *datasets* — trabalha apenas sobre as estruturas já carregadas em memória, evitando passagens sucessivas por eles.

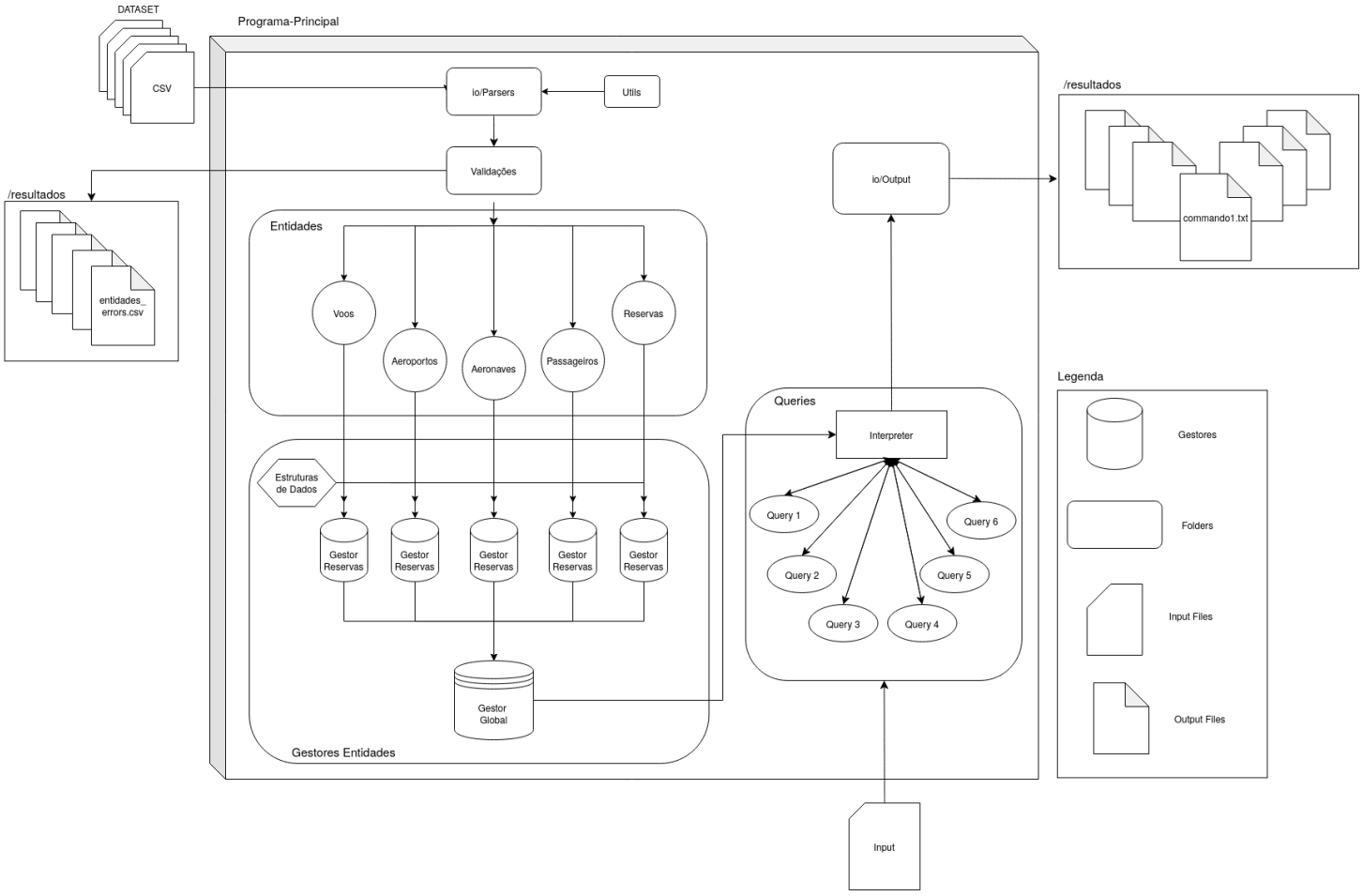


Figura 1: Sistema Implementado

2.2. Discussão

2.2.1. Fluxo do Programa

A função programa_principal coordena o fluxo do programa desde a leitura dos argumentos até à libertação final de recursos.

Inicialmente, inicializamos o Gestor Global, que centraliza os gestores de cada entidade. Continuamos com a opção de utilizar uma *Hash Table* (da biblioteca *glib*) em cada gestor (ver a exceção na parte das otimizações), o que nos assegura complexidade O(1) no acesso a qualquer entidade através do seu ID. Foram adicionadas mais estruturas de dados em cada gestor, as quais foram utilizadas como forma de tornar mais eficiente a realização das *queries* (a justificação das escolhas de cada estrutura está presente na parte onde ocorre a explicação das *queries*).

O processamento dos ficheiros *CSV* recorre a um parser genérico que recebe como argumento a função de leitura específica de cada entidade. Isto permitiu-nos evitar repetição de código e centralizar a lógica de abertura de ficheiros e tratamento de erros.

Nesta fase, optámos por manter a utilização da flag is_used nos parsers, ainda que atualmente todas as entidades sejam necessárias. Esta decisão visa preservar a robustez e modularidade do código, permitindo desativar facilmente o carregamento de entidades específicas em caso de necessidade.

Após o carregamento, o módulo de *queries* é chamado: o interpretador lê o ficheiro de *input*, identifica o comando, chama a função da *query* e utiliza o modulo de *output* para colocar o resultado no respetivo ficheiro. Por fim, garantimos a limpeza total da memória através da função *free_gestor_global*, assegurando que o programa termina sem *memory leaks*.

2.2.2. *Queries*

A execução das queries visou encontrar o melhor equilíbrio entre o uso de memória e a rapidez de execução. Para isso, optamos pelo pré-processamento dos dados, realizando o máximo de cálculos e organização durante o parse, o que minimiza as operações no momento da chamada das queries e, consequentemente, diminui o seu tempo de resposta.

2.2.2.1. *Query* 1

Listar o resumo de um aeroporto, consoante o identificador recebido por argumento.

A realização desta *query* foi direta. Utilizando a estrutura de dados adotada nos gestores — uma *Hash Table* — o acesso à informação de qualquer aeroporto pelo seu identificador é feito em tempo constante (O(1)). Para responder aos requisitos da segunda fase, acrescentámos dois campos à entidade aeroporto para contabilizar os passageiros que aterram e partem. À medida que fazíamos o *parse* das reservas, fomos atualizando estes valores nos respetivos aeroportos. Desta forma, a resposta à *query* resume-se a uma pesquisa na *hash table*, sem necessidade de cálculos adicionais no momento, preservando a complexidade O(1).

2.2.2.2. *Query* 2
Top N aeronaves com mais voos realizados com parâmetro opcional [manufacturer]

A implementação desta *query* procurou minimizar o consumo adicional de memória. A solução passou por adicionar um campo de contagem de voos diretamente na estrutura da entidade aeronave. Durante o *parse* do *dataset* de voos, este contador é incrementado sempre que a aeronave correspondente realiza um voo válido. Para facilitar o acesso e ordenação, foi adicionado um *GPtrArray* ao gestor de aeronaves, preenchido à medida que as entidades são criadas. Esta estrutura armazena apenas apontadores para as entidades existentes, evitando duplicação de dados e evitando um grande aumento no uso de memória. O gestor mantém uma *flag is_sorted* que indica se o *array* de aeronaves já foi ordenado por número de voos decrescente.Esta abordagem permite que a ordenação ocorra apenas na primeira execução da *query* 2. Caso a *query* 2 não seja chamada, evita-se completamente o custo da ordenação.

Para gerar a resposta, o processo depende da presença do filtro de fabricante: quando não existe filtro, percorrem-se simplesmente as primeiras N aeronaves do *GPtrArray* ordenado; quando o filtro está presente, percorre-se o *GPtrArray* sequencialmente até encontrar N aeronaves do fabricante especificado ou até esgotar todas as aeronaves disponíveis. Assim, depois de estar ordenado o *GPtrArray*, no pior caso, apenas percorremos as aeronaves todas (O(A), onde A é o número de aeronaves).

2.2.2.3. *Query* 3
Listar o aeroporto com mais partidas entre 2 datas

Para a *query* 3, adotamos uma estratégia semelhante à da *query* 2. Durante o carregamento do *dataset* de voos, cada data de partida é inserida num *GArray* associado ao aeroporto de origem correspondente. As datas são armazenadas como inteiros no formato AAAAMMDD, o que permite reduzir o espaço ocupado em memória e acelerar as operações de comparação. No gestor de aeroportos, foi criado um *GPtrArray* que facilita a iteração sobre todos os aeroportos e permite ordenar os *arrays* de datas de partida de cada um.

Tal como na *query* 2, o gestor mantém uma *flag is_sorted* que, neste caso, indica se os *arrays* de datas de cada aeroporto já foram ordenados. Desta forma, a ordenação ocorre apenas na primeira invocação da *query* 3, evitando ordenar várias vezes e também evita ordenar quando não é necessário. Com esta abordagem o cálculo do número de voos entre 2 datas fica eficiente: através de pesquisa binária, determina-se a posição da primeira data superior ou igual ao limite inferior do intervalo e a posição da última data inferior ou igual ao limite superior. O número de partidas é então obtido pela diferença entre os índices encontrados, acrescida de uma unidade.

Para identificar o aeroporto com mais partidas, percorremos todos os aeroportos, calculamos o número de partidas no intervalo através do método descrito, e mantemos registo do aeroporto com o maior valor encontrado. Assim a complexidade desta *query* é O(A . log(D)) onde A é o número de aeroportos e D é o número de datas de partida de cada aeroporto. Ao substituir uma procura linear (O(N)) por uma pesquisa binária (O(log N)), conseguimos garantir tempos de resposta bastante melhores, independentemente do grande crescimento do número de voos registados em cada aeroporto no novo dataset.

2.2.2.4. *Query* 4

Qual o passageiro que esteve mais tempo no top 10 de passageiros que mais gastaram em viagens durante um período?

Esta query revelou-se de dificuldade acrescida, sendo que foi desenvolvida de forma a ser o mais rápida e eficiente possível. Assim sendo, dividimos a sua resolução em 3 partes:

1) Durante a leitura do *dataset* de reservas, utilizamos uma estrutura temporária (*week_expenses*) para agregar os gastos. Trata-se de um array dinâmico (*GPtrArray*) onde cada índice corresponde a uma semana do ano (calculada através da diferença de dias em relação a uma data de referência). Em cada posição do *array*, existe uma *Hash Table* que mapeia o identificador do passageiro (*document_number*) ao total gasto nessa semana. Desta forma, à medida que processamos as reservas, somamos os valores monetários na semana correspondente e do respetivo passageiro. Isto é feito em tempo constante pois acedemos à semana em tempo constante (O(1)) e o acesso à *hash table* para atualizar os gastos também é feito em tempo constante.

2) Após a conclusão do *parse*, convertemos a estrutura de dados "pesada" (*week_expenses*) numa estrutura otimizada para consulta (top10_by_week). Para cada semana, percorremos a *Hash Table* de gastos e determinamos os 10 passageiros com maior despesa. Para o fazer eficientemente, recorremos a uma *GSequence* (estrutura de dados da biblioteca *glib* que mantém uma lista ordenada com complexidade de inserção logarítmica). À medida que iteramos sobre os passageiros de uma semana inserimos o passageiro na sequência de forma ordenada pelo valor gasto (decrescente). Se a sequência ultrapassar 10 elementos, removemos o último (o que gastou menos entre os analisados). No final deste processo, extraímos apenas os identificadores dos 10 passageiros para um *array* de inteiros simples (*int*)**.**
À medida que fazemos isto libertamos imediatamente a memória das Hash Tables e das GSequences. A estrutura final que permanece em memória (top10_by_week) é apenas uma matriz de inteiros (*int*), o que reduz drasticamente a ocupação de RAM comparativamente a manter todos os gastos de todos os passageiros. Este processo tem um custo inicial linear face ao número de passageiros (O(N)), dado que a manutenção de uma lista ordenada de tamanho fixo é reduzido (10 elementos) é uma operação de custo praticamente constante.

3) Com os tops semanais já pré-calculados, a função get_most_times_in_top_10 torna-se eficiente. O algoritmo começa por converter o intervalo de tempo recebido em índices de semanas para percorrer a matriz top10_by_week. Utilizamos uma *Hash Table* auxiliar para contagem de frequência (as *keys* são os *document_number* e o os *values* são a frequência no *top* 10). Mantemos variáveis que registam o passageiro com maior contagem até ao momento. A cada atualização na tabela de *hash*, verificamos imediatamente se aquele passageiro ultrapassou o máximo atual ou se, em caso de empate, possui um ID menor. Desta forma, assim que terminamos de percorrer as semanas, temos o resultado pronto a retornar. Esta operação tem um custo de O(W), onde W corresponde ao número de semanas compreendidas no intervalo de tempo solicitado uma vez que processamos um número constante de elementos por semana (apenas 10) e que as operações de inserção e consulta na *Hash Table* possuem complexidade média O(1).

2.2.2.5. *Query* 5
Top N companhias aéreas com mais tempo de atraso médio por voo

A resolução desta *query* exige o cálculo dos tempos médios de atraso por companhia aérea. Para tal, tal como na query anterior dividimos a resolução da *query* em diferentes etapas.

1) À medida que o *parser* processa o ficheiro de voos, identificamos aqueles que sofreram atrasos. Para estes casos, utilizamos uma *Hash Table* (atraso_companhia) presente no gestor de voos, onde a chave é o nome da companhia aérea e o valor é uma estrutura auxiliar (AirlineStats), que vão sendo atualizadas durante o *parse* . Esta estrutura armazena apenas dois acumuladores: o tempo total de atraso (em minutos) e o número de voos atrasados. Ao atualizar estes valores em tempo real, evitamos a necessidade de guardar uma lista individual de todos os atrasos, poupando memória.

2) Após a conclusão do *parse*, invocamos a função *build_most_delayed*. Esta função percorre a *Hash Table* de agregação e transfere os dados para um *GPtrArray* (atraso_companhia_order). Durante esta transferência, calculamos a média de atraso para cada companhia e libertamos a memória da *Hash Table* original, mantendo apenas a informação essencial no *array*. Para otimizar a alocação de memória, o tamanho do *array* é pré-definido com base no contador *num_companhias* existente no gestor dos voos (que é utilizado durante o *parse* dos voos), garantindo que não existe desperdício de espaço alocado.

3) Para responder à *query*, o *array* necessita de estar ordenado por média de atraso (decrescente). Desta forma, tal como nas *queries* anteriores, aplicamos uma estratégia de ordenação. O gestor mantém uma *flag is_sorted* que indica o estado do *array*. A ordenação (que tem um custo O(N log N)) só é executada na primeira vez que a *query* 5 é solicitada (se não for chamada não ordena). Nas execuções subsequentes, se a *query* for chamada novamente o sistema deteta que os dados já estão ordenados. Para dar a resposta à *query* apenas é necessário percorrer os primeiros elementos N (número pedido na *query*) elementos deste *array*, pelo que a complexidade é O(N) (podendo ser O(A), ou A é o número de *airlines* no caso de $N > A$).

2.2.2.6. *Query* 6
Listar o aeroporto de destino mais comum para passageiros de uma determinada nacionalidade

A estratégia para esta *query* assemelha-se à da *Query* 4, baseando-se no pré-processamento intensivo para garantir respostas praticamente imediatas. Inicialmente, ponderámos utilizar uma *Hash Table* de *GTrees* (árvores balanceadas) para poupar memória, visto que as árvores apenas alocam nós à medida que necessário. Contudo, após testes de desempenho, optámos por converter a estrutura para uma *Hash Table* cujo o seu *value* é outra *Hash Tables*, priorizando a complexidade temporal O(1) em detrimento de uma ligeira poupança espacial. Os passos inerentes à resolução desta query são os seguintes:

1) Durante o processamento das reservas, é estabelecida a relação entre a nacionalidade do passageiro e o aeroporto de destino. Para armazenar estes dados temporariamente, optou-se por uma *Hash Table* cujo campo *value* é outra *Hash Table*. A tabela exterior utiliza a nacionalidade

como chave e o seu valor corresponde a uma segunda *Hash Table*. Nesta tabela interna, a chave é o código *IATA* do aeroporto e o valor é um contador de ocorrências. Esta organização permite incrementar a frequência de qualquer destino para uma determinada nacionalidade com uma complexidade média de O(1).

2) Após a conclusão da leitura do *dataset* das reservas, a estrutura aninhada contém a contagem total de todos os destinos por nacionalidade. Para otimizar a consulta, e apenas armazenar os resultados que importam para a execução da interrogação, executa-se a função build_q6. Esta função itera sobre cada nacionalidade e percorre a respetiva sub-tabela de destinos para identificar o aeroporto com o maior número de ocorrências (aplicando o critério de desempate lexicográfico quando necessário). O resultado final é guardado numa nova *Hash Table* simples (*hash_nacionalidades*), onde a chave é a nacionalidade e o valor é a estrutura contendo apenas o *IATA* mais frequente para aquela nacionalidade e a respetiva contagem. Concluída esta transferência, a estrutura aninhada inicial é libertada da memória.

3) Com os dados obtidos na tabela final, a função get_q6 realiza uma consulta direta numa *Hash Table*. A função recebe a nacionalidade por argumento e verifica a sua existência na *hash_nacionalidades*. Caso exista, retorna o aeroporto previamente calculado. Dado que se trata de um acesso direto a uma tabela de *hash*, a operação apresenta uma complexidade temporal de O(1), não dependendo do volume total de dados processados anteriormente.

2.2.3. Otimizações

Nesta fase, implementámos diversas otimizações focadas tanto na redução do tempo de execução quanto na eficiência do consumo de memória. Uma das principais estratégias adotadas para minimizar a memória utilizada foi a implementação de uma *String Pool*. Face à dimensão dos *datasets* identificou-se que atributos como nacionalidades e nomes de companhias aéreas repetem-se inúmeras vezes. Para evitar a alocação redundante de memória para estes valores repetidos, recorreu-se à criação de um módulo string_pool.c que se baseia numa *GHashTable* para definir o tipo *String_Pool*. Esta estrutura centraliza o armazenamento de *strings*, permitindo uma única alocação para uma mesma *string*, reduzindo drasticamente a quantidade de memória do programa.

Além disso, evitou-se armazenar dados que não são utilizados pelas *queries* existentes. Por exemplo, as reservas não foram guardadas na sua totalidade, uma vez que os atributos necessários para a resolução das *queries* foram extraídos durante o processo de *parsing*, evitando assim o armazenamento de elementos desnecessários. Optou-se por manter o código referente a esta entidade comentado, de forma a facilitar eventuais alterações futuras. No que diz respeito às melhorias no tempo de execução, demos prioridade à otimização das funções que são chamadas milhões de vezes. Identificámos que as funções de validação e tratamento de strings, por serem executadas para cada linha de cada ficheiro CSV, representavam um custo elevado, tendo sido os principais alvos destas otimizações.

Uma outra melhoria foi a substituição da GHashTable no gestor de aeroportos por um GPtrArray. Como os códigos IATA são compostos por apenas 3 letras, o número total de combinações possíveis é reduzido (26^3=17.576), permitindo converter o seu id num inteiro entre zero e o valor máximo. Esta abordagem permite eliminar os custos associado ao cálculo de funções de hash e à consequente diminuição de comparações entre strings devido a colisões na GHashTable, uma vez que o id passou a um inteiro. Foi escolhido o GPtrArray uma vez que

garante acessos à memória instantâneos (O(1)) e mantém a flexibilidade de gestão dinâmica da GLib. Esta alteração é praticável dado que o consumo de memória para mapear todo o universo de aeroportos é insignificante para os recursos atuais.

2.2.4. Testes

2.2.4.1. Programa Testes

No programa de testes mantivemos a estrutura base desenvolvida na fase anterior, tendo este na mesma a capacidade de mostrar o tempo médio de parse por entidade, a memória utilizada em cada parse, o tempo médio de cada query, o número de queries erradas assim como a primeira linha que está mal e ainda o tempo e memória total utilizadas no programa. A melhoria introduzida nesta fase foi ao nível da identificação de erros: implementámos um sistema de comparação visual que, em caso de discrepância, imprime no terminal a linha esperada e a obtida a comparando-as com cores.

2.2.4.2. Especificações

As máquinas dos 3 elementos do grupo possuem as seguintes especificações:

Computador 1:
- OS: Ubuntu 24.04.2 LTS - kernel 6.14.0-35-generic
- CPU: 13th Gen Intel(R) Core(TM) i7-13700H
- Memoria: 16GB DDR5-4800MHz

Computador 2:
- OS: Pop! OS 22.04 LTS - kernel 6.9.3-76
- CPU: 13th Gen Intel® Core™ i5-1335U
- Memoria: 16GB DDR5-5200MHz

Computador 3:
- OS: ArchOs - 6.18.1--arch1-1
- CPU:9th Gen Intel(R) Core(TM) i7-9850H
- Memoria: 16GB DDR4-3200MHz

2.2.4.3. Resultados dos Testes

Para a análise de desempenho, mantivemos a metodologia estabelecida na primeira fase. Os testes foram realizados utilizando o dataset maior e com erros, num ambiente igual para todos (computadores ligados à corrente, recém-iniciados e sem processos de fundo). Para garantir a estabilidade e fiabilidade dos dados, efetuámos execuções de aquecimento da cache antes de registar a média de três medições finais. As tabelas com os resultados obtidos nestes testes encontram-se nos anexos.

Ao analisar os resultados, notamos que o tempo de execução do programa varia significativamente de computador para computador. Por exemplo, o Computador 2 foi o mais rápido (cerca de 9,5 segundos), enquanto o Computador 3 demorou quase 14 segundos. Por outro lado, o uso de memória foi muito parecido em todos os casos, rondando sempre os 668 MB no total. Isto prova que o consumo de memória depende apenas da forma como organizamos os nossos dados e não do computador onde o programa está a ser executado.

2.2.5. Programa Interativo

O modo interativo foi implementado recorrendo à biblioteca *ncurses*, o que permitiu criar uma UI (*User Interface*) visualmente organizada. A arquitetura deste módulo foi desenhada para ser modular: abstraímos a lógica visual em componentes genéricos (como janelas de *input*, menus de seleção e visualizadores de texto), o que facilitou a construção das diferentes vistas do

programa. Funcionalmente, o sistema guia o utilizador desde a seleção do *dataset* até à execução das *queries*, oferecendo funcionalidades essenciais de usabilidade, tais como um sistema de paginação para resultados extensos e ajuda contextual sobre os parâmetros de cada comando. Imagens referentes a este programa encontram-se nos anexos.

2.2.6. Métodos de Encapsulamento

O encapsulamento foi implementado para restringir o acesso aos dados exclusivamente ao módulo que os gere. Para tal, utilizámos estruturas opacas, mantendo as definições nos ficheiros de implementação (.c) e expondo apenas as declarações na interface através de *getters* e *setters*.

Relativamente à troca de dados, e para evitar a quebra de encapsulamento inerente à partilha de ponteiros, optámos por uma estratégia de isolamento total de memória: tanto os *getters* como os *setters* recorrem à função *strdup* (nos casos que envolvem alocação dinâmica) para duplicar a informação, impedindo assim o acesso direto à memória interna da estrutura. Foi também utilizada a estratégia *const char ** nas situações em que o atributo era exigido muitas vezes (da ordem dos milhões de acessos) de forma a evitar cópias excessivas que aumentavam o tempo de execução. Para garantir que não havia um comprometimento da segurança dos dados, aliámos esta prática à *flag* de compilação *-Werror=discarded-qualifiers*. Desta forma, o próprio compilador impede a criação do executável caso exista qualquer tentativa de alterar estes dados protegidos.

Estas abordagens garantem que qualquer manipulação externa ocorre sobre uma cópia ou ponteiro imutável, preservando a integridade dos dados internos e prevenindo erros de memória por acessos indevidos.

2.2.7. Dificuldades Sentidas

Os maiores obstáculos enfrentados na segunda fase assemelharam-se aos da primeira fase: com a gestão manual de memória e a evolução da arquitetura do sistema. A resolução de *memory leaks* foi particularmente exigente devido à complexidade das novas estruturas de dados adotadas para maximizar o desempenho. Adicionalmente, a necessidade de otimizar os tempos de execução forçou uma reestruturação frequente do código base. Este processo de refatorização implicou múltiplas adaptações nos módulos de interação entre entidades, gerando desafios na manutenção da estabilidade do sistema durante a transição para estruturas mais eficientes.

3. Conclusão

Em suma, consideramos que o projeto foi concluído com sucesso, tendo cumprido integralmente os requisitos propostos, tal como se pode ver pela plataforma de testes onde conseguimos passar com nota máxima em todos os testes.

Nesta etapa, fomos forçados a repensar várias estruturas de dados para lidar com a escala do dataset mantendo a eficiência. Mecanismos como a String Pool e o pré-cálculo de resultados foram mecanismos essenciais para que as novas funcionalidades operassem de forma fluida e passassem em todos os testes.

Este projeto constituiu, assim, uma experiência fundamental para consolidar a aplicação de princípios de modularidade e encapsulamento em C, resultando num sistema que consideramos eficiente.

4. Anexos

Parse	Computador 1 (s)	Computador 2 (s)	Computador 3 (s)
Parser Voos	4.393451	3.471763	4.628434
	4.525492	3.55220	4.669772
	4.483046	3.495483	4.682773
Parser Aeroportos	0.023923	0.012279	0.023787
	0.022929	0.012637	0.023547
	0.023232	0.013596	0.023491
Parser Aeronaves	0.006893	0.004108	0.008706
	0.006902	0.004292	0.008738
	0.006574	0.004333	0.008652
Parser Passageiros	1.408981	1.027420	1.457188
	1.320464	1.041984	1.862026
	1.232451	1.040937	1.458692
Parser Reservas	5.102456	4.633833	5.686479
	4.803252	4.649230	5.466482
	4.989234	4.647082	5.366287
Média Parser Voos	4.467330	3.506484	4.660326
Média Parser Aeroportos	0.023361	0.012837	0.023608
Média Parser Aeronaves	0.006790	0.004244	0.008699
Média Parser Passageiros	1.320632	1.036780	1.592633
Média Parser Reservas	4.964981	4.643382	5.506416

Tabela 1: Tempos de *parse* dos *datasets* nos 3 computadores.

Parser	Computador 1 (KB)	Computador 2 (KB)	Computador 3 (KB)
Parser Voos	335796	335662.67	335512
Parser Aeroportos	3828	6124	6216
Parser Aeronaves	800	910	1224
Parser Passageiros	159520	159530.67	159490.67
Parser Reservas	167520	167082.67	167905.33

Tabela 2: Uso médio de memória durante o *parse* dos *datasets* referente às 3 execuções

Query	Computador 1 (ms)	Computador 2 (ms)	Computador 3 (ms)
Q1	0.053562	0.023887	0.688294
	0.0467824	0.016485	0.802482
	0.0434673	0.013494	0.050899
Q2	0.057432	0.056492	0.319024
	0.060932	0.046970	0.307044
	0.061924	0.049401	1.580294
Q3	6.323456	5.516239	10.310352
	6.932617	5.699031	10.392139
	6.643425	5.584476	10.827576
Q4	0.014023	0.011847	0.018340
	0.013902	0.011476	0.018042
	0.014219	0.011479	0.020778
Q5	0.019243	0.010680	0.019768
	0.020292	0.010687	0.019095
	0.018234	0.010759	0.022540
Q6	0.006890	0.005160	0.011480
	0.006349	0.005067	0.009531
	0.005903	0.005255	0.014491
Média Q1	0.047937	0.017956	0.513891
Média Q2	0.060096	0.050954	0.735454
Média Q3	6.633166	5.599915	10.51002
Média Q4	0.014048	0.0116	0.019053
Média Q5	0.019256	0.010709	0.020467
Média Q6	0.006381	0.005160	0.011834

Tabela 3: Tempos de execução das *queries* nos 3 computadores.

	Computador 1	Computador 2	Computador 3
Tempo Total (s)	11.12213	9.492677	13.88735
Memória Total (KB)	667464	668528	669096

Tabela 4: Tempo médio total de execução e média da memória total utilizada nas 3 execuções.



Figura 2: Programa Interativo Menu



Figura 3: Programa Interativo Seleção Query